Pthreads Data Parallelism: Parallel Histogram

Cheolwon Park (Student ID: 2024270616)
Korea University
scott9020@korea.ac.kr

1 Implementation

Three TODO sections in `histogram.c` were completed to build a parallel histogram using Pthreads data parallelism. The worker function `hist_worker` zeroes its local histogram via `memset`, then iterates over its assigned chunk (`data[start..end]`) incrementing the corresponding bin. Since each thread writes only to its own `local_hist` array, no synchronization is needed. In `histogram_parallel`, the main thread divides the data into `data_size/nthreads` chunks (the last thread absorbs the remainder), launches each thread with `pthread_create`, then calls `pthread_join` on all threads and merges their local histograms into the global result by summing each bin across all threads.

File	Modification
`histogram.c`	Completed `hist_worker()`, thread creation loop, and join-merge loop

2 Theoretical Background

2.1 Why Local Histograms Instead of a Shared One?

If all threads incremented a shared `global_hist[data[i]]` directly, multiple threads could perform read-modify-write on the same bin simultaneously, causing a race condition. For example, two threads reading the same count of 5 might both write back 6, losing one increment. Protecting the shared array with a mutex on every increment would serialize the entire counting phase, destroying parallelism. Using per-thread local histograms eliminates all write conflicts; the only shared resource is the read-only `data` array, which requires no protection. This is the same principle as `partial_sum[id]` from Lab 2 — giving each thread its own output space removes the need for synchronization.

2.2 Speedup and Amdahl's Law

The following table shows measured performance with 10,000,000 elements:

Threads	Time (sec)	Speedup
Sequential	0.0061	1.00x
1	0.0038	1.60x
2	0.0030	2.05x
4	0.0024	2.57x
8	0.0016	3.91x

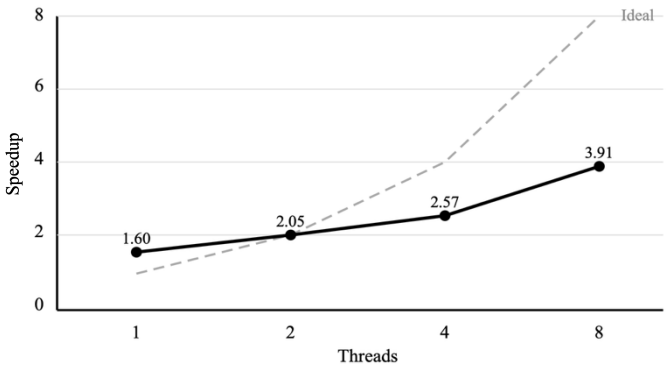

Figure 1. Speedup vs. thread count (N = 10M). Dashed line = ideal linear speedup

The 1-thread case shows speedup above 1.0x due to cache warming: the sequential baseline runs first on the 40MB data array (cold cache), loading it from main memory. All subsequent parallel runs benefit from the data already being partially resident in CPU cache. Speedup grows sub-linearly, consistent with Amdahl's Law: $S = \frac{1}{f+\frac{1-f}{N}}$. Solving for the serial fraction f from the 8-thread result:

$$3.91 = \frac{1}{f+\frac{1-f}{8}} \Rightarrow f + \frac{1-f}{8} = 0.2558 \Rightarrow \frac{7}{8}f = 0.1308 \Rightarrow f \approx 0.15(15\%).$$

This serial fraction accounts for the merge phase, thread creation/join overhead, and data generation.

2.3 Thread Creation/Join Overhead

To quantify thread overhead, the program was tested with varying data sizes (8 threads):

Data Size	Seq. Time	8-Thread Time	Speedup
1,000	< 1 μs	~ 100 μs	< 0.01x
10,000	~ 6 μs	~ 100 μs	0.06x
100,000	~ 56 μs	~ 100 μs	0.56x
10,000,000	6,100 μs	1,600 μs	3.91x

With 1,000 elements, each thread processes only ~125 elements, yet each `pthread_create`/ `pthread_join` pair incurs tens of microseconds of kernel scheduling overhead. The fixed thread management cost dominates, making parallel execution significantly *slower* than sequential. Speedup only exceeds 1.0x beyond roughly 100K elements, where per-thread work outweighs the creation and joining cost. This illustrates that Amdahl's Law must account for fixed overhead: at small *N* the serial cost dominates regardless of thread count.

2.4 Comparison with Lab 2 (Parallel Array Sum)

Similarities. Both assignments use the same data-parallel pattern from Silberschatz Ch. 4.2: the main thread partitions the input into equal chunks, spawns worker threads that each compute a partial result independently via the create-loop / join-loop structure, then merges after joining. Both use per-thread storage to avoid synchronization — `partial_sum[id]` in Lab 2, `local_hist[NUM_BINS]` here — and both are embarrassingly parallel with no inter-thread communication during the computation phase.

Differences. The shape of each thread's result differs. In Lab 2, each thread produces a single scalar (partial sum), making merge a simple *N*-way addition. In the histogram, each thread produces a 100-element array, so merge iterates over all bins per thread. Despite this, merge cost remains negligible because the bin count (100) is tiny relative to the data size (10M). Additionally, array summation has sequential memory access (one addition per element), while histogram counting involves random-access writes (`local_hist[data[i]]++`) that can cause more cache misses. Despite this, the embarrassingly parallel structure ensures strong scaling in both cases.